Fable 5.1 — Trading System Master Prompt

Deep Strategy Research • Existing MVP Audit • MT5 + NinjaTrader 8 • Data Architecture • Rule Management • News Engine • Relationship Discovery • ML Roadmap • Phased Production Build

How to use this document: Give this prompt to Fable 5.1 together with the trading-strategy PDF. The PDF containing the strategy rules is a separate input and is the current strategy specification. Fable must inspect the existing codebase before proposing implementation changes.

CORE DIRECTIVE

You are acting as a senior quantitative researcher, systematic trading architect, ML engineer, trading-systems engineer, data engineer, and software architect. The project is already in an MVP stage. Your job is to transform the existing MVP into a research-driven, testable, maintainable, and eventually production-grade system without blindly rebuilding it.

Do not start coding immediately. First audit the repository, understand the strategy PDF, conduct research, identify weaknesses, design the target architecture, compare it with the existing architecture, and produce a phased implementation plan. Only then should implementation begin.

1. EXISTING CODEBASE — MANDATORY AUDIT

```
The project already contains approximately:

AI_Brain/
■■■ .claude/
■■■ .claude-flow/
■■■ .swarm/
■■■ backend/
■■■ data/
■■■ frontend/
■■■ Specs/
■■■ task_pending/
■■■ tests/
■■■ venv/
■■■ .env
■■■ .gitignore
■■■ CLAUDE.md
■■■ requirements.txt
■■■ run.txt
■■■ ruVector.db
■■■ start_bot.bat

The exact contents may differ. Inspect the actual repository rather than relying on this tree.
```

Do NOT create a new project from scratch. Do NOT assume folders are empty. Do NOT replace technologies merely because another technology is more popular. Do NOT create duplicate systems if an existing component can be extended.

Inspect:
- backend
- frontend
- data
- Specs
- tests
- task_pending
- CLAUDE.md
- requirements.txt
- .env structure (never expose secrets)
- run.txt
- start_bot.bat
- .claude / .claude-flow / .swarm where relevant

- ruVector.db and its actual schema/use
- existing APIs
- existing strategy logic
- existing market-data integrations
- existing MT5 integration
- existing NT8 integration
- existing news integration
- existing dashboards
- logging/monitoring
- configuration
- test coverage
- scripts and entry points

```
Create an architecture map derived from the actual codebase. For each major component document:
Component → location → responsibility → inputs → outputs → dependencies → database interactions →
current status → problems → future role.
```

Classify existing components as:
- Working
- Partially working
- Experimental
- Unused
- Duplicated
- Broken
- Needs refactoring
- Retain
- Replace
- Remove

Do not make destructive changes during the audit.

2. STRATEGY PDF — SOURCE OF TRUTH

I will provide a PDF containing the current trading strategy and rules. Treat the PDF as the current specification, not as unquestionable truth.

First reconstruct the strategy precisely:
- markets/instruments
- asset classes
- sessions
- timeframes
- indicators
- entry conditions
- exit conditions
- stop loss
- take profit
- trailing/break-even
- position sizing
- risk rules
- exposure rules
- maximum trades
- drawdown/loss limits
- filters
- execution assumptions
- timing/timezone rules
- discretionary decisions
- exceptions

Separate every statement into:
1. Existing rule from PDF
2. Proposed improvement
3. Research-backed recommendation
4. Engineering requirement
5. Experimental hypothesis
6. Open decision

Never silently change a rule.

Whenever the PDF is ambiguous, mark it as:
AMBIGUITY / NEEDS DECISION

Explain exactly what is ambiguous and what decision is required before coding.

3. CRITICAL STRATEGY AUDIT

Attack the strategy as if you were trying to prove it does NOT work. Investigate:
- overfitting
- curve fitting
- look-ahead bias
- data leakage
- survivorship bias
- selection bias
- unrealistic fills
- spread/slippage assumptions
- latency sensitivity
- repainting
- incomplete candles
- timezone errors
- session boundary errors
- news leakage
- correlated-position risk
- regime dependency
- parameter instability
- excessive rule complexity
- hidden discretion
- rules that cannot be objectively coded
- insufficient sample size

```
For every weakness use:
Problem → Why it matters → Evidence/research → Proposed solution → Validation test.
```

4. DEEP EXTERNAL RESEARCH

Conduct serious external research. Prefer academic papers, peer-reviewed research, exchange/broker/platform documentation, established quantitative-finance literature, and high-quality professional research over generic trading blogs.

Research, where relevant:
- walk-forward analysis
- out-of-sample testing
- Monte Carlo analysis
- parameter sensitivity
- stability testing
- market-regime detection
- volatility regimes
- liquidity regimes

- cross-asset correlation
- portfolio construction
- volatility targeting
- correlation-adjusted exposure
- execution quality
- spread/slippage modeling
- latency
- order types
- partial fills
- market impact
- event-driven backtesting

For every major recommendation explain whether it is:
Established research / Industry practice / Engineering recommendation / Hypothesis requiring testing.

If research conflicts, explain the disagreement.

5. TARGET ARCHITECTURE — DO NOT BLINDLY REBUILD

```
Design a target architecture that evolves the existing MVP. The desired conceptual separation is:

MT5 Adapter ■■
■→ Common Data/Event Layer → Common Strategy Engine
NT8 Adapter ■■ ↓
Rule Engine
Risk Engine
News Engine
Execution Engine
↓
MT5 / NT8 adapters

Trade + Market Events
↓
Data Store
↓
Analytics / Research / Features
↓
Statistics / ML
↓
Relationship Discovery
↓
Strategy Research
↓
New Strategy Version
```

Keep platform-specific integration separate from strategy logic. The same strategy specification should be executable against both MT5 and NT8 wherever the market/execution semantics permit it.

Explain which components belong in Python, C#/NinjaScript, MQL5, SQL, or other technologies. Optimize for reliability, simplicity, observability, and development speed—not technology fashion.

6. MT5 INTEGRATION

Research and design the most reliable MT5 integration. Evaluate:
- MQL5 Expert Advisor
- Python integration
- files
- database connectivity
- HTTP/REST
- WebSocket
- messaging
- local services

- appropriate broker/API constraints

Specify:
- market-data ingestion
- order submission
- order acknowledgements
- position synchronization
- fills
- slippage
- reconnect behavior
- duplicate-order prevention
- timestamps
- account/risk state
- error handling
- audit logs

Clearly identify what must run inside MT5 and what should run in the external common engine.

7. NINJATRADER 8 INTEGRATION

Research and design the most reliable NT8 integration. Evaluate:
- NinjaScript
- C#
- Add-ons
- local services
- APIs
- files
- database connectivity
- messaging

Specify:
- market-data ingestion
- order lifecycle
- fills
- positions
- execution events
- reconnect behavior
- synchronization
- duplicate-order protection
- timestamps
- account/risk state
- error handling
- audit logging

Keep NT8-specific code isolated behind an adapter boundary so the strategy is not coupled to NinjaTrader internals.

8. COMMON DATA MODEL

Design a normalized data model shared by MT5 and NT8. Capture, as applicable:
- tick
- bid
- ask
- last
- volume
- OHLC
- spread
- timestamp

- timezone
- symbol
- exchange
- session
- account
- order
- fill
- position
- trade

Every trade should be traceable to:
- unique trade ID
- strategy ID
- strategy version
- platform
- account
- asset
- direction
- entry/exit timestamps
- entry/exit prices
- stop/target
- quantity
- risk
- gross/net P&L;
- fees
- slippage
- MAE
- MFE
- holding time
- signal state
- market regime
- volatility
- spread
- session
- news state
- relevant feature values
- entry reason
- exit reason
- rule-set version

The system must be able to answer:
'Which exact rules, data, configuration, and model state generated this trade?'

9. DATABASE & STORAGE ARCHITECTURE

Do not automatically assume SQLite is enough. Evaluate:
- SQLite
- PostgreSQL
- TimescaleDB
- DuckDB
- Parquet
- object storage
- Redis
- message queues

Recommend storage separately for:
- raw market data
- normalized market data
- trades/orders/positions
- features
- research datasets
- experiments
- model artifacts/outputs
- configuration
- strategy versions
- audit logs
- live telemetry

Explicitly inspect the existing ruVector.db. Determine its actual purpose, schema, contents, and whether it should remain part of the AI/RAG knowledge layer. Do not assume it should become the authoritative trading database.

Propose an MVP storage architecture and a migration path to a larger system only when scale requires it.

10. RULE MANAGEMENT & VERSIONING

```
Rules must not be scattered throughout code. Design a versioned rule/configuration system.

Conceptually:
Strategy
→ Strategy Version
→ Rule Set
→ Risk Configuration
→ Asset Configuration
→ Session Configuration
→ News Configuration
→ Execution Configuration
```

A rule change must create a new version rather than silently changing historical behavior.

Every trade, backtest, and research experiment must retain:
- strategy version
- rule version
- configuration version
- dataset version
- model version if applicable

Design validation so invalid or contradictory configurations are rejected before deployment.

11. TRADE RELATIONSHIPS & DISCOVERY

Design a system capable of studying relationships among:
- assets
- trades
- sessions
- market regimes
- volatility
- liquidity
- news
- execution
- signal states
- trade sequences
- portfolio exposure

Examples:
Asset A × Asset B
Asset × Strategy

Asset × Regime
Session × Strategy
News × Strategy
Volatility × Strategy
Setup × Outcome
Previous Trade Outcome × Next Trade
Holding Time × Outcome
Spread × Outcome
Slippage × Outcome

```
Do not confuse correlation with causality. Distinguish:
Correlation → Predictive relationship → Possible causal mechanism.
```

Use robust statistical validation and multiple-testing controls where appropriate. Avoid mining thousands of relationships and treating the best-looking one as truth.

12. FEATURE STORE

Design a feature architecture covering:
Price:
- returns
- momentum
- range
- ATR
- realized volatility
- trend strength

Market structure:
- highs/lows
- breakouts
- distances
- compression/expansion

Cross-asset:
- correlation
- relative strength
- spreads
- lead/lag

Time:
- hour
- session
- day of week
- market-open/close proximity

Execution:
- spread
- slippage
- latency
- liquidity proxies

News:
- minutes until event
- minutes since event
- event importance
- affected asset/currency

- event category

Determine which features are useful, redundant, unstable, or likely to introduce leakage/noise. Do not build a feature simply because it is easy to calculate.

13. NEWS ENGINE

Current rule:
STOP TRADING 15 MINUTES BEFORE relevant news and remain stopped for 15 MINUTES AFTER the news.

Implement this as a centralized, auditable rule rather than duplicating news checks inside multiple strategies.

```
Design:
News Event → normalized timestamp → affected assets → importance → pre-buffer → post-buffer → blackout
decision
```

Research:
- high/medium/low impact events
- central-bank events
- CPI
- NFP
- FOMC
- interest-rate decisions
- GDP
- employment
- unexpected/unscheduled news
- asset-specific news

Do not silently change the ±15-minute rule. Instead, propose experiments such as:
Version A: ±15m
Version B: high impact ±30m, medium impact ±15m
Version C: asset/event-specific windows

Validate alternatives out-of-sample before adoption.

Define behavior when the news feed is unavailable, delayed, contradictory, or revised. Trading must fail safely.

14. BACKTESTING

Design a trustworthy backtesting framework that models:
- bid/ask
- spread
- commission
- slippage
- latency
- partial fills
- trading hours
- news blackout
- position limits
- risk limits

Compare bar-based, tick-based, and event-driven backtesting. Recommend the appropriate level for this strategy based on actual rules and execution sensitivity.

Backtest results must be reproducible using:
Strategy Version + Rule Version + Dataset Version + Parameters + Code Version + Environment.

15. EXPERIMENT FRAMEWORK

Every experiment should have:
Experiment ID
Hypothesis
Strategy Version
Dataset Version
Parameters
Train period
Validation period
Test period
Metrics
Results
Conclusion
Decision

Never overwrite previous experiments. Preserve negative results. The research system must support reproducibility and comparison across strategy versions.

16. PERFORMANCE & RISK ANALYTICS

Track:
- total return
- CAGR where meaningful
- Sharpe
- Sortino
- Calmar
- profit factor
- expectancy
- win rate
- average win/loss
- maximum drawdown
- recovery factor
- risk of ruin
- MAE
- MFE
- holding time
- monthly/daily returns
- consecutive wins/losses
- return distribution
- skew
- kurtosis
- drawdown duration

Also analyze prop-firm-style constraints where relevant:
- daily loss
- maximum drawdown
- equity vs balance drawdown
- floating loss
- exposure
- correlated positions
- rule violations

Do not optimize solely for net profit.

17. ML / AI ROADMAP

Do not add ML because it sounds advanced. First establish a strong rule-based baseline.

Evaluate in sequence:
1. Rule-based strategy
2. Rule-based + statistical filters
3. Rule-based + ML filter
4. Adaptive models only if justified

Potential targets:
- probability of profitable trade
- expected return
- probability of stop-out
- expected drawdown
- regime classification
- position-size adjustment
- exit decisions

Use time-series-safe validation, walk-forward training, strict leakage prevention, drift monitoring, and reproducible datasets. Compare every ML model against the non-ML baseline.

Prefer interpretable/statistically stable models when they perform similarly to more complex models.

18. LIVE SYSTEM & EXECUTION

```
Design:
Market Data
→ Normalizer
→ Strategy Engine
→ Signal Validation
→ Risk Engine
→ News Engine
→ Execution Engine
→ MT5/NT8 Adapter
→ Broker/Exchange
→ Trade Events
→ Database
→ Analytics/Research
```

Include:
- structured logging
- monitoring
- health checks
- kill switch
- global and per-strategy risk limits
- duplicate-order prevention
- idempotency
- reconnect handling
- clock synchronization
- data-quality checks
- audit trail

Define clear authority for risk decisions. A platform adapter must not be able to bypass the centralized risk policy.

19. FAILURE & RECOVERY

Analyze:
- MT5 disconnect
- NT8 disconnect
- internet failure
- broker failure
- missing ticks
- duplicate ticks
- delayed ticks

- timestamp errors
- news feed failure
- database failure
- duplicate orders
- partial execution
- strategy crash
- machine restart
- clock drift
- API failure
- corrupt data

```
For every failure define:
Detection → Protection → Recovery → Logging → Operator visibility.
```

Design for fail-safe behavior, especially around risk, news blackout, order state, and position synchronization.

20. DASHBOARD / UI

Use the existing frontend where possible. Do not create a parallel dashboard without first auditing what exists.

Design live views for:
- positions
- P&L;
- risk
- exposure
- news blackout
- platform health
- market-data health
- strategy status
- execution status

Design analytics for:
- equity
- drawdown
- asset
- session
- setup
- regime
- volatility
- news
- execution quality

Design relationship exploration:
Asset × Strategy
Asset × Regime
Session × Strategy
News × Strategy
Volatility × Strategy
Setup × Outcome
Trade Sequence × Outcome

Every displayed metric should have a defined calculation and source.

21. TESTING

Design:
- unit tests
- rule tests

- integration tests
- adapter tests
- data-quality tests
- backtest validation
- risk-limit tests
- news-blackout tests
- failure/recovery tests
- paper-trading tests
- live reconciliation tests

Create tests that attempt to violate every risk and news rule. Test exact boundary conditions such as:
- exactly 15 minutes before news
- exactly at news time
- exactly 15 minutes after news
- timezone/session transitions
- reconnect during an open position
- duplicate execution events.

22. DEVELOPMENT PHASES

Build in phases. You may modify the ordering if research shows a better dependency structure.

```
PHASE 0 — Existing-system + strategy specification
Deliverables: repository audit, strategy normalization, ambiguity list, architecture map.

PHASE 1 — Data foundation
Deliverables: normalized schemas, ingestion, data validation, identifiers, timestamps, storage.

PHASE 2 — Research/backtesting foundation
Deliverables: reproducible datasets, realistic execution model, baseline metrics.

PHASE 3 — Strategy engine
Deliverables: exact implementation of current rules, rule tests, versioning.

PHASE 4 — MT5 adapter
Deliverables: market data, orders, fills, positions, synchronization, safety.

PHASE 5 — NT8 adapter
Deliverables: equivalent integration with platform-specific execution semantics.

PHASE 6 — Centralized risk + news engine
Deliverables: risk enforcement, ±15-minute blackout, safe failure behavior.

PHASE 7 — Analytics + relationship research
Deliverables: trade analytics, cross-asset analysis, regime analysis, execution analytics.

PHASE 8 — Robustness research
Deliverables: walk-forward, out-of-sample, Monte Carlo, sensitivity, stress tests.

PHASE 9 — ML experimentation
Deliverables: baseline comparison, leakage-safe experiments, model monitoring only if justified.

PHASE 10 — Paper trading
Deliverables: live shadow mode, reconciliation, execution comparison.

PHASE 11 — Controlled live deployment
Deliverables: small exposure, kill switch, alerts, strict risk limits.

PHASE 12 — Production hardening
Deliverables: monitoring, CI/CD, disaster recovery, auditability, versioned deployments.
```

23. TECHNOLOGY DECISIONS

Evaluate the existing stack first. Recommend changes only when justified.

Potential technologies to evaluate:
Core: Python, C#, MQL5, SQL
Data: PostgreSQL, TimescaleDB, DuckDB, Parquet, SQLite

Messaging: Redis, RabbitMQ, Kafka, ZeroMQ, WebSockets
ML: scikit-learn, XGBoost/LightGBM, PyTorch
Infrastructure: Docker, CI/CD, Kubernetes only if justified

```
For each technology decision provide:
Need → Options → Tradeoffs → Recommendation → Why now / Why later.
```

Optimize for:
Reliability + simplicity + observability + reproducibility + development speed.

24. REPOSITORY & SPECIFICATION EVOLUTION

Use the existing Specs/ directory as a specification layer rather than creating a second source of truth.

```
Propose an organization such as:
Specs/
■■■ strategy/
■■■ architecture/
■■■ integrations/
■■■ risk/
■■■ news/
■■■ data/
■■■ research/
■■■ decisions/

Do not create all of these blindly. First inspect the current Specs directory and adapt it.
```

The original strategy PDF should remain an immutable reference. A normalized machine-readable specification should be derived from it, version controlled, and linked to experiments and trades.

25. FINAL RESEARCH REPORT

Produce a comprehensive report with:
1. Executive Summary
2. Existing System Audit
3. Existing Strategy Reconstruction
4. Ambiguities
5. Critical Weaknesses
6. Research Findings
7. Recommended Strategy Improvements
8. Existing vs Target Architecture
9. MT5 Architecture
10. NT8 Architecture
11. Common Strategy Engine
12. Data Architecture
13. Database Design
14. Rule Management
15. News Architecture
16. Trade Relationship Engine
17. Feature Architecture
18. Backtesting
19. ML Roadmap
20. Dashboard
21. Monitoring
22. Failure Handling
23. Testing
24. Development Roadmap
25. Technology Decisions
26. Risks
27. Open Questions

28. Final Recommended Architecture

For every significant recommendation include evidence and clearly label the confidence/evidence strength.

26. PRIORITIZATION

Create a final table:
Improvement | Expected Impact | Difficulty | Risk | Evidence Strength | Priority

Priority:
P0 Critical
P1 High
P2 Medium
P3 Future

Then provide:
FIRST 10 THINGS TO BUILD — exact order
THINGS NOT TO BUILD YET — with reasons
KEY ARCHITECTURAL DECISIONS
KEY RESEARCH EXPERIMENTS
KEY OPEN QUESTIONS

27. NON-NEGOTIABLE PRINCIPLES

1. Do not blindly rebuild the existing MVP.
2. Do not silently modify strategy rules.
3. Do not code before the audit and design.
4. Do not optimize only for backtest profit.
5. Do not use future information.
6. Do not allow ML to bypass risk controls.
7. Do not mix platform-specific code with core strategy logic.
8. Do not overwrite historical experiments.
9. Do not treat correlation as causation.
10. Do not assume SQLite, vector DBs, Kafka, Kubernetes, or ML are automatically necessary.
11. Do not expose secrets from .env.
12. Make every live trade auditable.
13. Make every strategy version reproducible.
14. Fail safely when market/news/data infrastructure is unavailable.
15. Prefer the simplest architecture that can satisfy the actual requirements.

28. FINAL QUESTION YOU MUST ANSWER

Finish the research with a brutally practical answer:

'If I were building this system from the current MVP today, what exactly should I build first, what should I research first, what should I postpone, and what architecture should I commit to only after validation?'

Give the answer as an ordered execution plan with dependencies and acceptance criteria.

OUTPUT QUALITY CHECKLIST

Before presenting your final work, verify that you have actually inspected the existing repository and that every architectural recommendation accounts for what already exists. Explicitly identify anything you could not inspect.

Check	Required outcome
Existing MVP	Audited before proposing replacement
Strategy PDF	Fully reconstructed; ambiguities identified
Research	Evidence-backed and source-linked
MT5	Adapter and execution architecture defined
NT8	Adapter and execution architecture defined
Data	Normalized cross-platform model defined
Database	Storage choices justified
Rules	Versioned and auditable
News	±15m rule centralized and fail-safe
Relationships	Statistical discovery without false causality
ML	Only after baseline + leakage-safe validation
Backtesting	Realistic execution assumptions
Testing	Unit → integration → paper → live
Roadmap	Phased with acceptance criteria
MVP discipline	Unnecessary complexity explicitly postponed

End state: The goal is a research-driven systematic trading platform where Strategy Rules → Data → Backtesting → Execution → Trade History → Analytics → Relationship Discovery → Research → Modeling → Improved Strategy → New Strategy Version → Validation → Deployment, with complete reproducibility and auditability.